

Ling 185A: Assignment 8

Dylan Bumford, Spring 2025

due: Fri, June 6

1 Incremental Parsing

1. In this problem we will consider the garden-path sentence “The bird watched from the tree sang” from the perspective of **incremental parsing**. Here again is the little CFG from last week.

$S \rightarrow DP VP$	$N \rightarrow \text{nun, chef, baby, boss, bird, award, tree}$
$DP \rightarrow DP PS N$	$DP \rightarrow \text{Mary, John}$
$DP \rightarrow D NP$	$VT \rightarrow \text{met, saw, won, watched}$
$NP \rightarrow N (PP) (SRC) (ORC) (PRC)$	$VI \rightarrow \text{cried, left, sang}$
$VP \rightarrow VI (PP)$	$D \rightarrow \text{the}$
$VP \rightarrow VT DP (PP)$	$P \rightarrow \text{on, in, with, from}$
$SRC \rightarrow C VP$	$C \rightarrow \text{that}$
$ORC \rightarrow DP VT$	$PS \rightarrow \text{'s}$
$PRC \rightarrow VT PP$	$PP \rightarrow P DP$

- a. Give a **bottom-up** derivation of this sentence by successively applying SHIFT and REDUCE transitions. (Here, you only need to supply the configuration component of each row, not the transition type or production rule involved.)

```
gtbu :: [Config]
gtbu =
  [ ([], ["the", "bird", "watched", "from", "the", "tree", "sang"]) -- starting config
  -- , fill in the rest
  -- , of the steps
  , undefined
  ]
```

Now that you have a successful parse, put yourself in the shoes of a person who at first **mis-parses** the sentence in the way we have discussed. How would that attempted derivation have gone differently? Specifically, at what step in the derivation above could you have made a wrong turn, and what does that wrong turn consist in?

```
{-----
Answer in a sentence or two here
-----}
```

- b. Now, provide a **top-down** derivation of the same sentence using the rules PREDICT and MATCH. In principle, since all of the stack symbols here are *predictions*, they should all be barred, but don't worry about that for this exercise.

```
gttd :: [Config]
gttd =
  [ ([S], ["the", "bird", "watched", "from", "the", "tree", "sang"]) -- starting config
  -- , fill in the rest
  -- , of the steps
  , undefined
  ]
```

As before, consider how a **mis-parse** would have gone instead. At what step in the derivation could the wrong turn have happened, and what does that wrong turn consist in?

```
{-----  
Answer in a sentence or two here  
-----}
```

- c. Let's suppose that the human parsing system "backtracks" when it reaches a dead end configuration by going back a few steps to start off down a different path. And suppose that this backtracking can be detected in experiments where we follow the movements of people's eyes as they read (it can!). Explain how we could use our garden path sentence in such an experiment to decide whether humans use a bottom-up parser or a top-down parser (assuming, for this question, that these are the only two possibilities). Think about what the buffer looks like at the crucial point in each derivation.

```
{-----  
Answer in a sentence or two here  
-----}
```

2 Tree Automata

Have a look at `FTA.hs`. Along with the data types and machines introduced in class, there are some examples of concrete trees that you may want to use to test some of your work.

2.1 Finite Tree Automata Parsing

2. Write a function to parse a tree using a bottom-up tree automaton. This should return the list of states that a machine could end up in after consuming the tree from the leaves to the root using the provided transitions. With the functions we've written in class, and with enough careful thinking about types, this should be about a one-line list comprehension.

```
walkFTA :: Eq q => [Transition q] -> Tree String -> [q]  
walkFTA delta (Node c cs) = undefined
```

```
ghci> walkFTA (getDelta evenAsFTA) t14  
[E]  
ghci> walkFTA (getDelta polarityFTA) t20  
[Neu]
```

3. Write a function to check whether a given `FTA` can generate a given tree. Remember that in order for the tree to be grammatical, there must be at least one tree-shaped path through the machine that ends in an acceptable final state.

```
generates :: Eq q => FTA q -> Tree String -> Bool  
generates m t = undefined
```

```
ghci> generates polarityFTA t20  
True  
ghci> generates polarityFTA (Node "." [Node "anything" [], t20])  
False  
ghci> generates evenAsFTA t14  
True  
ghci> generates evenAsFTA (Node "a" [t14])  
False
```

2.2 Wh-in-situ dependencies

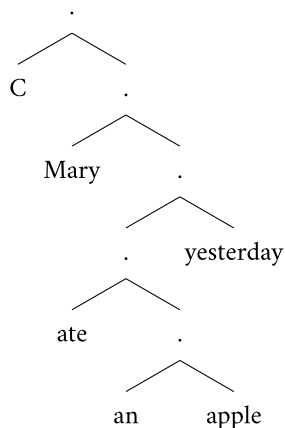
2.2.1 The basic pattern

In syntax classes you’ve probably talked about “wh-in-situ” languages (such as Japanese, Mandarin, Turkish, Hindi, and many others), where wh-words that move to the front of a question in languages like English instead stay in their base position. So in a language which is wh-in-situ but otherwise like English — let’s call it Whenglish — a question formed from (1a) by asking about the object position would look almost the same as (1a), as shown in (1b), rather than (1c) where we see the effect of wh-movement.

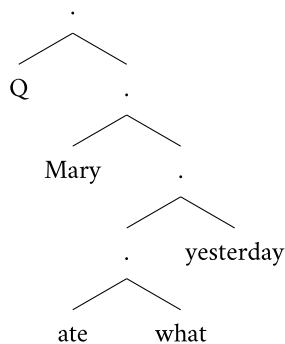
- (1) a. Mary ate an apple yesterday
 b. Mary ate what yesterday
 c. what Mary ate yesterday

Let’s assume that declarative sentences begin with a declarative complementizer C, and interrogative sentences with an interrogative complementizer Q. In English, the Q complementizer attracts the embedded ‘wh’ word, but in Whenglish, it does not, as depicted in the trees below.

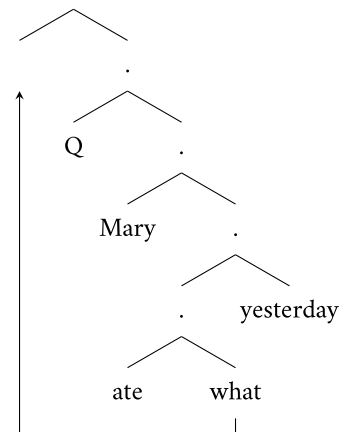
(wh)english



whenglish



english



Evidence for the idea that there’s a distinguished Q complementizer comes from the fact that it’s pronounced overtly in many languages. For example, in Japanese it’s pronounced ‘no’; see (2), corresponding to (1a) and (1b). The C complementizer in the declarative is unpronounced, like in English. The Q complementizer must co-occur with a wh-word, and vice-versa, as shown in (3).

- (2) a. John-wa ringo-o tabeta
 John-TOP apple-ACC ate
 ‘John ate an apple’

- b. John-wa nani-o tabeta no
 John-TOP what-ACC ate Q
 ‘What did John eat?’

- (3) a. *John-wa ringo-o tabeta no
 John-TOP apple-ACC ate Q

- b. *John-wa nani-o tabeta
 John-TOP what-ACC ate

From this point of view, the *dependency* between a wh-word and a Q complementizer is similar to the dependency between an NPI and its licensing negative element, which we discussed in class. Specifically, here's a complete set of rules for well-formed trees in Whenglish:

- Every wh-word must be c-commanded by a Q complementizer.
 - A single Q can license multiple wh-words that it c-commands, just like a single negation can license multiple NPIs. When this happens it produces “multiple questions”, like the English ‘Who ate what?’.
 - Every Q must c-command at least one wh-word. This is different from the pattern with NPIs; a negative element need not c-command an NPI.
 - Any leaf node with a string in the list `qWords` is a Q complementizer.
 - Any leaf node with a string in the list `whWords` is a wh-word.
 - Other leaf nodes are allowed to have any string in the list `plainWords`.
 - Branching nodes should have exactly two daughters and the label “.”.
4. Define a (bottom-up) Finite Tree Automaton `wh1FTA` that enforces these rules. You can choose whatever you like as the state type of the machine. You might like to define a custom type for this purpose, like we did for `polarityFTA`. (If you do, make sure you include `deriving (Eq, Show)`.)

```
-- choose a type for q and uncomment this type declaration:
-- wh1FTA :: FTA q
wh1FTA = undefined

ghci> generates wh1FTA tree_1a
True
ghci> generates wh1FTA tree_1b
True
ghci> generates wh1FTA tree_3a
False
ghci> generates wh1FTA tree_3b
False
ghci> generates wh1FTA (mrg (lf "Q") (lf "what"))
True
ghci> generates wh1FTA (mrg (lf "Q") (lf "Mary"))
False
ghci> generates wh1FTA (mrg (lf "C") (lf "what"))
False
ghci> generates wh1FTA (mrg (lf "Q") (mrg (lf "what") (lf "why")))
True
ghci> generates wh1FTA (mrg (lf "C") (mrg (lf "what") (lf "why")))
False
ghci> generates wh1FTA (mrg (lf "Q") (mrg (lf "John") (lf "what")))
True
ghci> generates wh1FTA (mrg (mrg (lf "Q") (lf "John")) (lf "what"))
False
```

2.2.2 Island effects

You've probably also learned in syntax classes about the way wh-movement out of certain kinds of structures, called **islands**, is disallowed. Here we'll concentrate on **adjunct islands**.

Adjuncts include relative clauses, as bracketed (4a), and 'because'-clauses, as in (4b).

- (4) a. Mary likes the person [that bought books as a gift]
 b. Mary laughed [because John bought books as a gift]

As a result, sentences that attempt to move a wh-word out of these constituents are ungrammatical¹

- (5) a. *What does Mary like the person that bought ____ as a gift for John?
 b. *Why does Mary like the person that bought books ____?
- (6) a. *What did John laugh [because Mary bought ____ as a gift]?
 b. *Why did John laugh [because Mary bought books ____]?

Interestingly, the configurations that block wh-movement in these English examples can also disrupt the dependency between Q and in-situ wh-words. Here's some relevant data from Mandarin Chinese, showing adjuncts in brackets.

- (7) *ni zui xihuan [weishenme mai shu de] ren ?
 you most like why buy book DE person
 'Why do you like the person that bought books?'
- (8) *ta [zai Lisi weishenme mai shu yihou] shengqi le ?
 he at Lisi why buy book after angry LE
 'Why did he get angry after Lisi bought books?'

And here's something along the same lines from Japanese.

- (9) *Mary-ga [John-ni naze hon-o ageta] hito-ni atta no ?
 Mary-NOM John-DAT why book-ACC gave man-DAT meet Q
 Why did Mary meet the man that gave books to John?

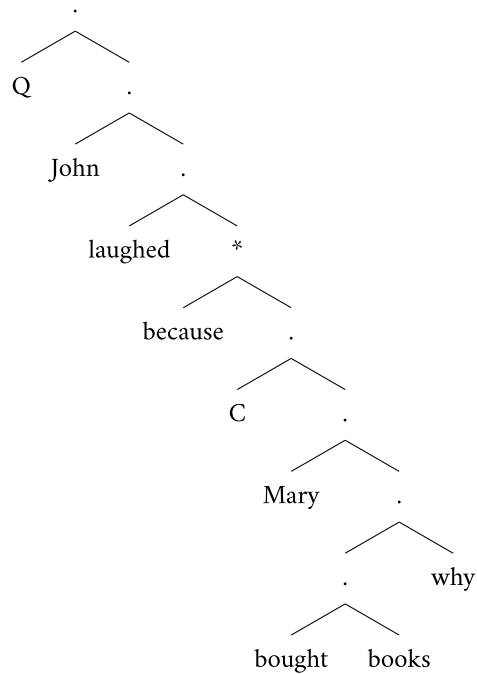
So we'll assume that Whenglish follows the same pattern.

5. Define a (bottom-up) Finite Tree Automaton **wh2FTA** that enforces the same basic dependency pattern between Q and wh-words as in **wh1FTA**, and additionally guarantees that the licensing Q for a wh-word is not separated from it by an adjunct island boundary. You will probably find that you can use the same states you used for **wh1FTA** here, though you don't have to. We will use "*****" as a non-leaf symbol to identify (the root nodes of) adjuncts. Specifically:

- All of the rules from **wh1FTA** above still apply, *except that* branching nodes can now be labeled either "." or "*****".
- A constituent whose root node has the label "*****" is an adjunct.
- If a wh-word is contained within an adjunct, then it can only be licensed by a Q complementizer that is also within that adjunct. And still, every Q complementizer must license at least one wh-word in this way.

Here's a tree that should **not** be generated by **wh2FTA**, for example. It is defined for you in **FTA.hs** as **tree_13**.

¹ The interrogatives in (5b) and (6b) are trying to get at questions that would be answered by "as a gift", where the 'why' asks for the reason that the books were bought, not the reason for the liking/laughing.



```

-- choose a type for q and uncomment this type declaration:
-- wh2FTA :: FTA q
wh2FTA = undefined

```

```

ghci> generates wh2FTA tree_13
False
ghci> generates wh2FTA (Node "." [Node "Q" [], Node "*" [Node "C" [], Node "why" []]])
False
ghci> generates wh2FTA (Node "." [Node "Q" [], Node "." [Node "C" [], Node "why" []]])
True
ghci> generates wh2FTA (Node "." [Node "Q" [], Node "*" [Node "C" [], Node "why" []]])
False
ghci> generates wh2FTA (Node "." [Node "C" [], Node "*" [Node "Q" [], Node "why" []]])
True

```